

Obstáculos para una llama

Una llama quiere viajar por el Altiplano Andino. Tiene un mapa del altiplano en forma de cuadrícula de $N \times M$ celdas cuadradas. Las filas del mapa están numeradas de 0 a $N - 1$ de arriba a abajo, y las columnas están numeradas de 0 a $M - 1$ de izquierda a derecha. La celda en la fila i y columna j ($0 \leq i < N, 0 \leq j < M$) se expresa como (i, j) .

La llama estudió el clima del altiplano y descubrió que en cada fila todas las celdas tienen la misma **temperatura** y en cada columna todas las celdas tienen la misma **humedad**. La llama te ha proporcionado dos vectores de enteros T y H de longitud N y M respectivamente. Donde $T[i]$ ($0 \leq i < N$) indica la temperatura de las celdas de la fila i , y $H[j]$ ($0 \leq j < M$) indica la humedad de las celdas de la columna j .

La llama también estudió la flora del altiplano y observó que una celda (i, j) está **libre de vegetación** si y sólo si su temperatura es mayor que su humedad, formalmente $T[i] > H[j]$.

La llama sólo puede atravesar el altiplano siguiendo **caminos válidos**. Un camino válido es una secuencia de celdas distintas que satisfacen las siguientes condiciones:

- Cada par de celdas consecutivas en el camino tienen un lado en común.
- Todas las celdas del camino están libres de vegetación.

Tu tarea es responder Q preguntas. Para cada pregunta, se te asignan cuatro números enteros: L , R , S , y D . Debés determinar si existe un camino válido tal que:

- El camino comienza en la celda $(0, S)$ y termina en la celda $(0, D)$.
- Todas las celdas del camino se encuentran entre las columnas L y R , incluidas.

Se garantiza que tanto $(0, S)$ como $(0, D)$ estén libres de vegetación.

Detalles de implementación

La primera función que debés implementar es:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : un arreglo de longitud N que especifica la temperatura en cada fila.
- H : un arreglo de longitud M que especifica la humedad en cada columna.

- Esta función se llama exactamente una vez para cada caso de prueba, antes de cualquier llamada a `can_reach`.

La segunda función que debés implementar es:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : números enteros que describen una pregunta.
- Esta función se llama Q veces para cada caso de prueba.

Esta función debe devolver `true` si y solo si existe un camino válido desde la celda $(0, S)$ a la celda $(0, D)$, de manera que todas las celdas en el camino se encuentren entre las columnas L y R , incluidas.

Restricciones

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ para cada j tal que $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Ambas celdas $(0, S)$ y $(0, D)$ están libres de vegetación.

Subtareas

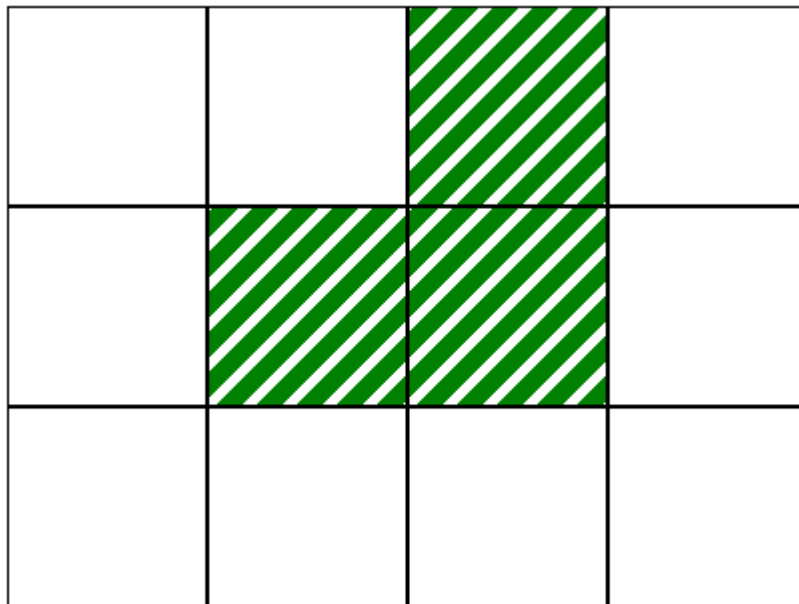
Subtarea	Puntos	Restricciones adicionales
1	10	$L = 0, R = M - 1$ para cada pregunta. $N = 1$.
2	14	$L = 0, R = M - 1$ para cada pregunta. $T[i - 1] \leq T[i]$ para cada i tal que $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ para cada pregunta. $N = 3$ y $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ para cada pregunta. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ para cada pregunta.
6	17	Sin restricciones adicionales.

Ejemplo

Considera la siguiente llamada:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

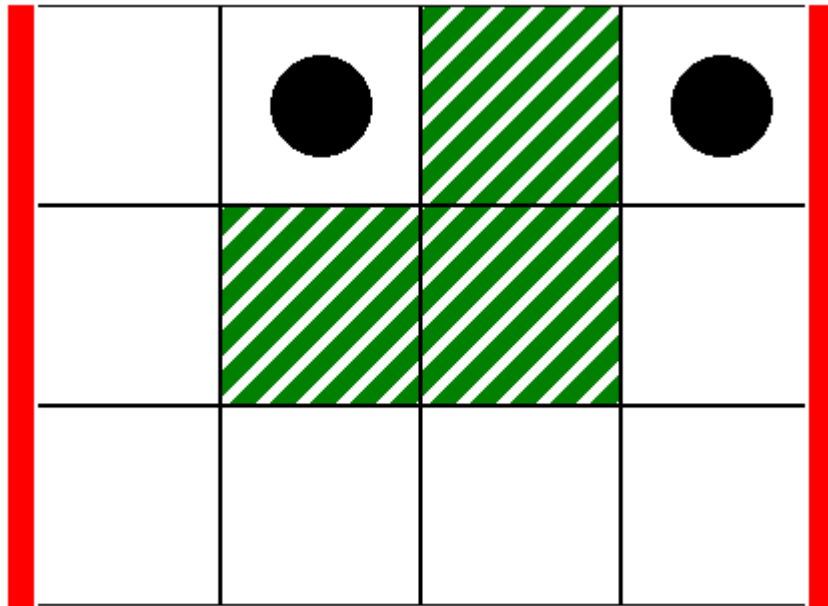
Esta se corresponde con el mapa de la siguiente imagen, donde las celdas blancas están libres de vegetación:



Como primera pregunta, considera la siguiente llamada:

```
can_reach(0, 3, 1, 3)
```

Esta se corresponde con la situación de la siguiente imagen, donde las líneas verticales gruesas indican el rango de columnas desde $L = 0$ a $R = 3$, y los discos negros indican las celdas iniciales y finales:



En este caso, la llama puede llegar desde la celda $(0,1)$ a la celda $(0,3)$ a través del siguiente camino válido:

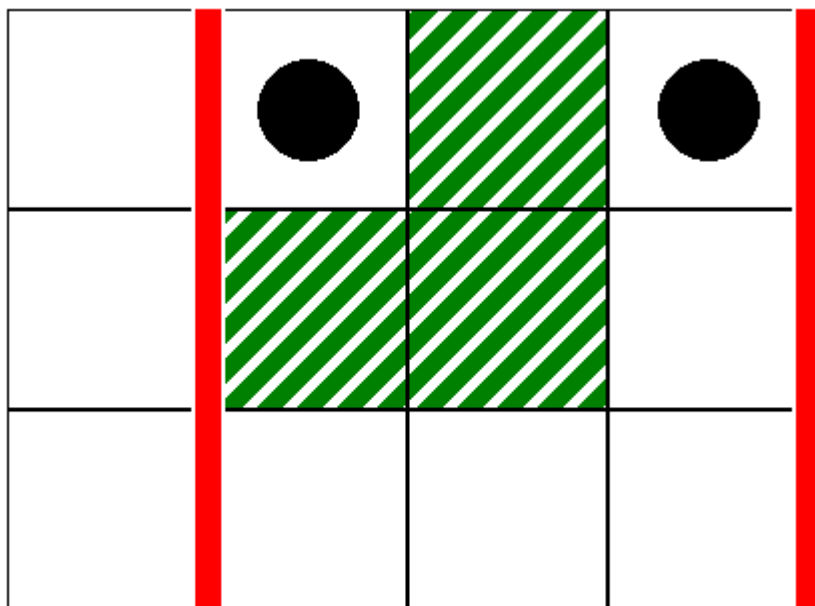
$(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)$

Por lo tanto, esta llamada debería devolver `true`.

Como segunda pregunta, considerará la siguiente llamada:

```
can_reach(1, 3, 1, 3)
```

Esta se corresponde con el escenario de la siguiente imagen:



En este caso, no hay un camino válido desde la celda $(0,1)$ hasta la celda $(0,3)$ tal que todas las celdas en el camino se encuentren entre las columnas 1 y 3, incluidas. Por lo tanto, esta llamada debería devolver `false`.

Evaluador local

Formato de entrada:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Donde, $L[k], R[k], S[k]$ y $D[k]$ ($0 \leq k < Q$) especifican los parámetros para cada llamada a `can_reach`.

Formato de salida:

```
A[0]
A[1]
...
A[Q-1]
```

Donde, $A[k]$ ($0 \leq k < Q$) es 1 si la llamada `can_reach(L[k], R[k], S[k], D[k])` devuelve `true`, y 0 en cualquier otro caso.